

1. Discuss the importance of visualizing the solutions of the N-Queens Problem to understand the placement of queens better. Use a graphical representation to show how queens are placed on the board for different values of N. Explain how visual tools can help in debugging the algorithm and gaining insights into the problem's complexity. Provide examples of visual representations for N = 4, N = 5, and N = 8, showing different valid solutions.

1.

+ Code

+ Text

```
def queen(n):
    board = [-1] * n
    def solve(r):
        if r == n:
            print(board)
            return True

        for c in range(n):
            if all(board[i] != c and abs(board[i]-c) != abs(i-r)
                  for i in range(r)):
                board[r] = c
                if solve(r+1):
                    return True
        return False
    solve(0)
    queen(4)
```

```
-- [1, 3, 0, 2]
```

2. Discuss the generalization of the N-Queens Problem to other board sizes and shapes, such as rectangular boards or boards with obstacles. Explain how the algorithm can be adapted to handle these variations and the additional constraints they introduce. Provide examples of solving generalized N-Queens Problems for different board configurations, such as an 8×10 board, a 5×5 board with obstacles, and a 6×6 board with restricted positions.

2.

```
n = 5
obstacles = [(2,2), (4,4)]
board = [-1] * n
def solve(r):
    if r == n:
        print(board)
        return True
    for c in range(n):
        if (r,c) not in obstacles:
            if all(board[i] != c and abs(board[i]-c) != abs(i-r)
                  for i in range(r)):
                board[r] = c
                if solve(r+1):
                    return True
    return False
solve(0)
```

```
[0, 2, 4, 1, 3]
True
```

3. Write a program to solve a Sudoku puzzle by filling the empty cells. A sudoku solution must satisfy all of the following rules: Each of the digits 1-9 must occur exactly once in each row. Each of the digits 1-9 must occur exactly once in each column. Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid. The '.' character indicates empty cells.

```

3.
def solve(board):
    for r in range(9):
        for c in range(9):
            if board[r][c] == '.':
                for n in range(1,10):
                    if n not in board[r] and \
                        all(board[i][c] != n for i in range(9)):
                        board[r][c] = n
                        if solve(board):
                            return True
                        board[r][c] = '.'
                return False
    return True

board = [
    [5,3,.,.,7,.,.,.],
    [6,.,.,1,9,5,.,.],
    [.,9,8,.,.,.,6,.],
    [8,.,.,6,.,.,3,],
    [4,.,8,.,3,.,.1],
    [7,.,.,2,.,.,6,],
    [.,6,.,.,2,8,.,],
    [.,.,4,1,9,.,5],
    [.,.,8,.,.,7,9]
]

solve(board)
for row in board:
    print(row)

```

```

... [5, 3, 1, 2, 7, 6, 4, 9, 8]
     [6, 2, 3, 1, 9, 5, 8, 4, 7]
     [1, 9, 8, 3, 4, 7, 5, 6, 2]
     [8, 1, 2, 7, 6, 4, 9, 5, 3]
     [4, 7, 9, 8, 5, 3, 6, 2, 1]
     [7, 4, 5, 9, 2, 8, 3, 1, 6]
     [9, 6, 7, 5, 3, 1, 2, 8, 4]
     [2, 8, 6, 4, 1, 9, 7, 3, 5]
     [3, 5, 4, 6, 8, 2, 1, 7, 9]

```

4. Write a program to solve a Sudoku puzzle by filling the empty cells. A sudoku solution must satisfy all of the following rules: Each of the digits 1-9 must occur exactly once in each row. Each of the digits 1-9 must occur exactly once in each column. Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid. The '.' character indicates empty cells.

```

3.
def solve(board):
    for r in range(9):
        for c in range(9):
            if board[r][c] == '.':
                for n in range(1,10):
                    if n not in board[r] and \
                        all(board[i][c] != n for i in range(9)):
                        board[r][c] = n
                        if solve(board):
                            return True
                        board[r][c] = '.'
                return False
    return True

board = [
    [5,3,.,.,7,.,.,.],
    [6,.,.,1,9,5,.,.],
    [.,9,8,.,.,.,6,.],
    [8,.,.,6,.,.,3,],
    [4,.,8,.,3,.,.1],
    [7,.,.,2,.,.,6,],
    [.,6,.,.,2,8,.,],
    [.,.,4,1,9,.,5],
    [.,.,8,.,.,7,9]
]

solve(board)
for row in board:
    print(row)

```

```

... [5, 3, 1, 2, 7, 6, 4, 9, 8]
     [6, 2, 3, 1, 9, 5, 8, 4, 7]
     [1, 9, 8, 3, 4, 7, 5, 6, 2]
     [8, 1, 2, 7, 6, 4, 9, 5, 3]
     [4, 7, 9, 8, 5, 3, 6, 2, 1]
     [7, 4, 5, 9, 2, 8, 3, 1, 6]
     [9, 6, 7, 5, 3, 1, 2, 8, 4]
     [2, 8, 6, 4, 1, 9, 7, 3, 5]
     [3, 5, 4, 6, 8, 2, 1, 7, 9]

```

5. You are given an integer array `nums` and an integer `target`. You want to build an expression out of `nums` by adding one of the symbols '+' and '-' before each integer in `nums` and then concatenate all the integers. For example, if `nums = [2, 1]`, you can add a '+' before 2 and a '-' before 1 and concatenate them to build the expression "+2-1". Return the number of different expressions that you can build, which evaluates to `target`.

5.

```
nums = [1,1,1,1,1]
target = 3
count = 0
def solve(i, total):
    global count
    if i == len(nums):
        if total == target:
            count += 1
        return
    solve(i+1, total+nums[i])
    solve(i+1, total-nums[i])
solve(0,0)
print(count)
```

5

6. Given an array of integers `arr`, find the sum of `min(b)`, where `b` ranges over every (contiguous) subarray of `arr`. Since the answer may be large, return the answer modulo $10^9 + 7$.

6.

```
a = [3,1,2,4]
total = 0
for i in range(len(a)):
    for j in range(i, len(a)):
        total += min(a[i:j+1])
print(total)
```

17

7. Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target. You may return the combinations in any order. The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different. The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

7.

```
a = [2,3,6,7]
target = 7
ans = []
def solve(i, total, x):
    if total == target:
        ans.append(x)
        return
    if total > target or i == len(a):
        return
    solve(i, total+a[i], x+[a[i]])
    solve(i+1, total, x)
solve(0,0,[])
print(ans)
```

```
[[2, 2, 3], [7]]
```

8. Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target. Each number in candidates may only be used once in the combination. The solution set must not contain duplicate combinations.

8.

```
a = [10,1,2,7,6,1,5]
target = 8
a.sort()
ans = []
def solve(i, total, x):
    if total == target:
        if x not in ans:
            ans.append(x)
        return
    if total > target:
        return
    for j in range(i, len(a)):
        solve(j+1, total+a[j], x+[a[j]])
solve(0,0,[])
print(ans)
```

```
[[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]
```

9. Given an array nums of distinct integers, return all the possible permutations. You can return the answer in any order.

9.

```
a = [1,2,3]
ans = []
def solve(x):
    if len(x) == len(a):
        ans.append(x)
        return
    for n in a:
        if n not in x:
            solve(x+[n])
solve([])
print(ans)
```

```
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
```

10. Given a collection of numbers, nums, that might contain duplicates, return all possible unique permutations in any order.

10.

```
from itertools import permutations
a = [1,1,2]
ans = list(set(permutations(a)))
print(ans)
```

```
[(1, 2, 1), (2, 1, 1), (1, 1, 2)]
```

11. You and your friends are assigned the task of coloring a map with a limited number of colors. The map is represented as a list of regions and their adjacency relationships. The rules are as follows: At each step, you can choose any uncolored region and color it with any available color. Your friend Alice follows the same strategy immediately after you, and then your friend Bob follows suit. You want to maximize the number of regions you personally color. Write a function that takes the map's adjacency list representation and returns the maximum number of regions you can color before all regions are colored. Write a program to implement the Graph coloring technique for an undirected graph. Implement an algorithm with minimum number of colors. edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] No. of vertices, n = 4

11.

```
edges = [(0,1),(1,2),(2,3),(3,0),(0,2)]
n = 4
color = [-1]*n
def solve(v):
    if v == n:
        return True
    for c in range(3):
        ok = True
        for a,b in edges:
            if a == v and color[b] == c:
                ok = False
            if b == v and color[a] == c:
                ok = False
        if ok:
            color[v] = c

            if solve(v+1):
                return True
            color[v] = -1
    return False
solve(0)
print(color)
```

[0, 1, 2, 1]

12. You and your friends are tasked with coloring a map using a limited set of colors, with the following rules: At each step, you can choose any region of the map that hasn't been colored yet and color it with any available color. Your friend Alice will then color the next region using the same strategy, followed by your friend Bob. You aim to maximize the number of regions you color. Given a map represented as a list of regions and their adjacency relationships, write a function to determine the maximum number of regions you can color. Write a program to implement the Graph coloring technique for an undirected graph. Implement an algorithm with minimum number of colors. edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] No. of vertices, n = 4, k = 3

12.

```
▶ edges = [(0,1),(1,2),(2,3),(3,0),(0,2)]
color = [-1]*4
k = 3
def solve(v):
    if v == 4:
        return True
    for c in range(k):
        ok = True
        for a,b in edges:
            if a == v and color[b] == c:
                ok = False
            if b == v and color[a] == c:
                ok = False
        if ok:
            color[v] = c

            if solve(v+1):
                return True

            color[v] = -1
    return False
print(solve(0))
print(color)
```

```
... True
[0, 1, 2, 1]
```

13. You are given an undirected graph represented by a list of edges and the number of vertices n . Your task is to determine if there exists a Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex. Write a function that takes the list of edges and the number of vertices as input and returns true if there exists a Hamiltonian cycle in the graph, otherwise return false. Example: Given edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2), (2, 4), (4, 0)] and $n = 5$

13.

```
edges = [(0,1),(1,2),(2,3),(3,0),(0,2),(2,4),(4,0)]
n = 5
path = [0]
def solve():
    if len(path) == n:
        return (path[-1],path[0]) in edges or (path[0],path[-1]) in edges
    for v in range(1,n):
        if v not in path:
            if (path[-1],v) in edges or (v,path[-1]) in edges:
                path.append(v)
                if solve():
                    return True
                path.pop()
    return False
print(solve())
```

False

14. You are given an undirected graph represented by a list of edges and the number of vertices n . Your task is to determine if there exists a Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex. Write a function that takes the list of edges and the number of vertices as input and returns true if there exists a Hamiltonian cycle in the graph, otherwise return false. Example: edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] and $n = 4$

14.

```
edges = [(0,1),(1,2),(2,3),(3,0),(0,2)]
n = 4
path = [0]
def solve():
    if len(path) == n:
        return (path[-1],0) in edges or (0,path[-1]) in edges
    for v in range(1,n):
        if v not in path:
            if (path[-1],v) in edges or (v,path[-1]) in edges:
                path.append(v)
                if solve():
                    return True
                path.pop()
    return False
print(solve())
```

True

15. You are tasked with designing an efficient coding to generate all subsets of a given set S containing n elements. Each subset should be outputted in lexicographical order. Return a list of lists where each inner list is a subset of the given set. Additionally, find out how your coding handles duplicate elements in S. A = [1, 2, 3] The subsets of [1, 2, 3] are: [], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]

15.

```
a = [1,2,3]
ans = []
def solve(i, x):
    if i == len(a):
        ans.append(x)
        return
    solve(i+1, x)
    solve(i+1, x+[a[i]])
solve(0, [])
print(ans)
```

```
[[], [3], [2], [2, 3], [1], [1, 3], [1, 2], [1, 2, 3]]
```

16. Write a program to implement the concept of subset generation. Given a set of unique integers and a specific integer 3, generate all subsets that contain the element 3. Return a list of lists where each inner list is a subset containing the element 3 E = [2, 3, 4, 5], x = 3, The subsets containing 3 : [3], [2, 3], [3, 4], [3,5], [2, 3, 4], [2, 3, 5], [3, 4, 5], [2, 3, 4, 5] Given an integer array nums of unique elements, return all possible subsets(the power set). The solution set must not contain duplicate subsets. Return the solution in any order.

16.

```
a = [2,3,4,5]
ans = []
def solve(i,x):
    if i == len(a):
        if 3 in x:
            ans.append(x)
        return
    solve(i+1,x)
    solve(i+1,x+[a[i]])
solve(0, [])
print(ans)
```

```
[[3], [3, 5], [3, 4], [3, 4, 5], [2, 3], [2, 3, 5], [2, 3, 4], [2, 3, 4, 5]]
```

17. You are given two string arrays words1 and words2. A string b is a subset of string a if every letter in b occurs in a including multiplicity. For example, "wrr" is a subset of "warrior" but is not a subset of "world". A string a from words1 is universal if for every string b in words2, b is a subset of a. Return an array of all the universal strings in words1. You may return the answer in any order.

17

```
words1 = ["amazon","apple","facebook","google","leetcode"]
words2 = ["e","o"]
ans = []
for word in words1:
    ok = True
    for x in words2:
        for c in x:
            if word.count(c) < x.count(c):
                ok = False
    if ok:
        ans.append(word)
print(ans)
```

```
['facebook', 'google', 'leetcode']
```